Native - iOS (Swift)

AI voice notes · Renovation photo intelligence
Architecture, prerequisites, and delivery plan

Purpose: This document is intended for **management and technical leadership** review. It summarizes proposed **architecture**, **iOS-side prerequisites** (Apple accounts, hardware, and tooling), and a **phased implementation** plan for a native iOS product that supports **voice-based notes** and **photo intelligence** in a home-renovation context. It assumes **one iOS developer** (no separate mobile team). **Cloud, AI vendors, and other backend prerequisites** belong in the **backend team's** companion document.

Scope of the solution: The first release targets **structured extraction from audio and images**—for example, turning a voice recording into an editable text note, and suggesting or inferring tags (such as before/after) from site photos, optionally combined with voice. The **mobile application** is responsible for capture and presentation; **speech recognition, image understanding, and large-language-model processing** run on **company-controlled backend services**, which can draw on the **broader application dataset** (projects, history, labels, and related media) when generating results.

---

1. High-level architecture

1.1 Guiding principles

- **Processing model: End-to-end flow:** the user records or captures content on the device → media is **uploaded to the backend** → the server performs speech-to-text, image analysis, and LLM steps as needed → a **structured response** is returned to the app for **review and editing**. **On-device** speech and ML are **not planned for v1**, so results can be informed by **organization-wide context** held on the server.
- **Context and how the product works:** The **backend** aggregates each customer's operational data—projects, sites, notes, photos, labels—so generated output can reflect **more than a single upload**. The **product** still defines business concepts (e.g. job site, project, before vs. after). The system uses AI to **propose** text and tags; **users confirm** final grouping, links, and labels.
- **Offline / sync: Out of scope for the initial release.** A later release may add offline queues and clearer "pending sync" experiences if required.

*Alternative approaches centered on strict on-device processing are **not** in scope for the first iteration; they may be revisited if product or policy requirements change.*

1.2 Role of the iOS application

The native app does **not** host the core AI workloads. Its responsibilities are: **capture** audio and images, **upload** them with the correct project and visit identifiers, **communicate with the API** for job status and results, **present** drafts and errors clearly, and allow the user to **edit and confirm** outcomes before persisting them via the server. The **backend** remains the **system of record** for notes, labels, and stored media.

- **Connectivity:** standard secure uploads (e.g. multipart or signed URLs); for longer-running jobs, the client may **poll** or receive **push** notifications when processing completes. **Background upload** may be deferred to a later phase if early releases remain foreground-oriented.

Figure — System context (trust boundaries)
The mobile app never holds third-party AI keys; only the backend calls external AI services.

```
flowchart TB
  subgraph field [Field / user]
    USER[User]
  end
  subgraph ios [iOS application]
    APP[Native app: capture and review UI]
  end
  subgraph backend [Company backend]
    API[API and job orchestration]
    DB[(Product datastore)]
    OBJ[(Media object storage)]
    API <--> DB
    API <--> OBJ
  end
  subgraph vendors [External AI vendors]
    AI[Speech-to-text, vision, LLM APIs]
  end
  USER <--> APP
  APP -->|TLS: upload media, call APIs| API
  API -->|Server-side credentials only| AI
```

1.3 End-to-end system flow (narrative)

For **voice-only**, **photo-only**, and **picture + voice** jobs, the **iOS app** follows the same pattern: **capture** → **upload** with project and visit context → **wait** for processing (poll or push as agreed) → **review** the structured draft → **save** through the API.

On the **backend**, the API ingests the upload, runs **speech-to-text** when audio is present, **vision or multimodal** analysis when images are present, then combines those signals in an **LLM** step that uses **tenant context** from the product datastore. Steps that do not apply to a given job (e.g. speech-to-text when there is no audio) are **skipped**. Credential handling matches the **trust boundaries** figure under **§1.2**.

1.4 Feature A — Comprehensive AI note from a voice clip

User flow (initial release)
Record → **upload audio and identifiers** to the backend → server runs **speech-to-text and LLM** steps using **tenant-wide data** → the app receives a **structured draft** → the user **reviews, edits, and saves**.

Figure — Voice note: sequence (typical happy path)

```
sequenceDiagram
  actor User
  participant App as iOS app
  participant API as Backend API
  participant Data as Product datastore
  participant AI as AI services

  User->>App: Start and stop recording
  App->>API: Upload audio and context IDs
  API->>Data: Load tenant context
  Data-->>API: Notes, metadata, labels
  API->>AI: Speech-to-text
```

```
AI-->>API: Transcript
API->>AI: LLM with transcript and context
AI-->>API: Structured draft
API-->>App: Draft payload
App->>User: Show draft for review
User->>App: Edit and confirm
App->>API: Persist confirmed note
API->>Data: Save final note
API-->>App: Acknowledgement
```

Processing (server-side after upload)

1. **Capture (device)** — standard iOS audio APIs; encoding as agreed with the API (e.g. AAC or WAV).
2. **Upload** — secure transfer (multipart upload or signed URL pattern), including project, visit, user, and locale where applicable.
3. **Backend** — speech-to-text, then LLM with **context retrieved** from the product datastore (e.g. recent notes, metadata, label vocabulary).
4. **Response** — a versioned **structured payload** (e.g. title, summary, bullet points, action items, optional confidence scores, optional transcript for display).
5. **Review (device)** — present the draft for editing; **persist** confirmed content through the API.
6. **Audit (backend)** — retain sufficient metadata (e.g. model version, job identifier) for operations and compliance, per policy.

*This path uses **audio** only on the vision side: speech-to-text and LLM run; vision steps are omitted. The **sequence diagram** above shows a typical happy path.*

Production practice: third-party AI services should be invoked from the **backend**, not from the mobile binary—so **credentials, usage limits, and access to organizational data** remain under company control.

1.5 Feature B — Image tagging & inference (renovation / before–after)

Goals

- Classify or suggest: **before vs after**, **room/area**, **defects** (optional), **materials visible** (optional).
- Support **multi-photo per site** and **pairing** before/after (user confirm).
- **Picture + voice**: one asset group — photo(s) + short voice; STT + vision **fused** in the LLM with a single structured result.

Inputs the system can combine (primarily on the server)

- **Visual: Backend** vision / **multimodal** model over uploaded images (and any stored thumbnails).
- **Metadata:** The client sends **date/time**, optional **GPS**, and **sequence** within a visit; the server joins **project and site history** from the product datastore.
- **User:** Selections such as before/after, **project**, and optional **room**—captured as fields on upload or in follow-up steps.
- **Audio:** A voice note attached to a photo batch → **server speech-to-text** → combined with image understanding in one LLM pass.
- **Heuristics:** Server-side rules (e.g. ordering within a visit) used as **soft priors**—still **suggestions** until the user confirms.

Recommended pattern: “AI suggests, human confirms”

1. The app **uploads** images and context identifiers to the API.
2. The backend runs **vision / multimodal and LLM** steps with the **same tenant-wide context** used for voice, returning a **structured result** (e.g. suggested phase, confidence, room estimate, labels).
3. The app **presents suggestions**; the user **accepts or corrects** them; outcomes are **stored via the API** for operational use and future quality improvements.

Figure — Photo tagging: sequence (typical path)

```
sequenceDiagram
  actor User
  participant App as iOS app
  participant API as Backend API
  participant Data as Product datastore
  participant AI as AI services

  User->>App: Take or select photos
  App->>API: Upload images and context IDs
  API->>Data: Load tenant context
  Data-->>API: Site and project history
  API->>AI: Vision or multimodal plus LLM
  AI-->>API: Structured suggestions
  API-->>App: Tags, phase, confidence
  App->>User: Show suggestions
  User->>App: Confirm or edit
  App->>API: Save final labels
  API->>Data: Persist
```

Picture + voice (multimodal)

- Upload: **image(s) + audio** in one job (or linked job ids); **all transcription on the server**.
- Backend: single pipeline that uses **image data, transcript, and organizational context** to produce tags, phase, captions, and related fields.
- Same **review** pattern as text notes.

*Photo-only jobs follow the **vision / multimodal and LLM** path with tenant context; **picture + voice** adds speech-to-text before that merge. The **sequence diagram** above illustrates a typical photo job.*

1.6 Security, privacy, and compliance (production expectations)

- **Transport security** (TLS) for all API traffic; additional hardening (e.g. certificate pinning) if required by security policy.
- **Encryption at rest** for object storage and application databases holding uploads and derived content.
- **Personally identifiable information** may appear in site imagery and audio; align on **retention**, **subprocessors**, and **customer agreements** (including use of external AI providers).
- **Authentication and authorization** (e.g. Sign in with Apple, enterprise identity, or tokens) so access to tenant data is **scoped per user and organization**.

---

2. Prerequisites (iOS only)

This section lists what **iOS development** needs from **procurement, IT, and finance** while **one person** owns the native client. **Server hosting, databases, AI providers, and backend secrets** are **not** covered here—

they should be defined and budgeted in the **backend team's** submission.

Apple developer access

- **Apple Developer Program** (approximately **USD 99 per year**) — required for installing builds on devices, **TestFlight**, and **App Store** submission. The **iOS Simulator** is included with Apple's development tools and is **not** billed separately.
- **Apple Developer / App Store Connect access** — ability to manage certificates, app identifiers, and releases (the **Account Holder** may be a manager; the developer still needs an appropriate role, e.g. **Admin** or **App Manager**, as decided by the organization).
- **Apple ID** (free) for the developer, added as a member of the company's **Apple Developer Program** team.

Hardware and devices

- **One Mac** that meets **current Xcode requirements** (Xcode runs only on macOS).
- **Physical iPhone(s)** for realistic microphone, camera, and performance testing (simulators alone are not sufficient for sign-off on capture features).

Engineering hygiene

- **Source control** (and **CI/CD** if the org uses it) for the iOS codebase, per organizational standards.

App Store release

- The same **Developer Program** enables **App Store Connect**; shipping to customers requires **privacy nutrition labels** and other **App Review** requirements for the client app.

Skills and interfaces

- The plan assumes **one engineer** comfortable with **Swift, SwiftUI, AVFoundation, and networking**. Delivery also depends on a **stable API contract** and environments from the **backend team** (see their document).

---

3. Implementation plan

Phases are **sequential**: **Phase 0** (foundations) → **Phase 1** (voice to AI note) → **Phase 2** (photo intelligence) → **Phase 3** (combined picture and voice) → **Phase 4** (hardening and scale). Timelines in each subsection are **indicative**.

Phase 0 — Foundations (indicative: 2–4 weeks)

- Application shell on iOS (SwiftUI), project structure, and integration points.
- Authentication approach agreed (Sign in with Apple optional for an early milestone).
- Basic **audio recording and playback**; **camera / photo selection**; screens consuming **server-provided** lists and detail where available.

Phase 1 — Voice to AI-generated note (core MVP) (indicative: 3–5 weeks)

- **Backend:** audio ingestion, speech-to-text, LLM pipeline, draft persistence; initial **context** from the product database (e.g. recent notes and project metadata, expanding over time).
- **iOS:** record → upload → await result → **review and edit** → save through the API.
- Formal **API contract** (versioned responses and error handling).
- User-visible handling of permissions, network failures, and low-quality or empty transcriptions.

Phase 2 — Photo intelligence: before/after and related tags (indicative: 3–6 weeks)

- Batch photo workflows tied to a **site visit**; upload to backend; same **context-aware** approach as voice.
- Backend **vision / multimodal** processing and structured outputs, reusing the Phase 1 context layer.
- User review of suggestions, overrides, and persistence via API.
- Optional: linking two photos explicitly as before/after pairs.

Phase 3 — Combined picture and voice; richer renovation outputs (indicative: 4–8 weeks)

- Unified processing for **images plus audio**; backend **speech, vision, and LLM** with shared organizational context.
- Expanded structured fields (e.g. materials, defects, client-ready summaries) as product requirements define.
- **Push notifications or polling** for long-running jobs; **background upload** if usage patterns require it.

Phase 4 — Hardening and scale

- Rate limiting, reliable retries, and idempotent uploads.
- Optional **administrative web experience** for B2B customers, if in scope.
- App Store **privacy** artifacts and **data lifecycle** (e.g. deletion) aligned with policy.

Ongoing in parallel

- **Product design:** language and flows appropriate to renovation crews; clear states while content is uploading or processing.
- **Legal and procurement:** iOS **App Store** privacy and disclosure obligations; **server-side** vendor and data terms should align with the **backend team's** documentation.

---

4. Summary

- **Architecture:** A native **SwiftUI** client **captures and uploads** content; **backend services** perform **speech-to-text, image understanding, and LLM** steps using **per-tenant organizational data**; users **review and confirm** results. **Version 1 does not rely on on-device speech recognition. Figures (this document): §1.2** — system context (trust boundaries); **§1.4** — voice note sequence (happy path); **§1.5** — photo tagging sequence (typical path). **§1.3** describes the same flows in prose.

- **Delivery:** Phases move from **foundations** through **voice-based notes**, **photo intelligence**, **combined media**, and **operational hardening** (see §3). Durations are **indicative** for a **single iOS developer** and subject to scope and dependency on the backend schedule.

- **Apple ecosystem: Developer Program** (annual fee) for device distribution and App Store; development **simulator** at no additional charge; microphone and camera **privacy disclosures** in the app as required. **On-device speech APIs** are unnecessary for the approach described here.

- **Backend and AI spend:**
  - Hosting, model vendors, and related **legal and compliance** work for server-side processing are **not covered in this document.**
  - Use the **backend team's** plan for budgets and vendors.

- **iOS dependencies: §2** lists **Apple, Mac, devices, and client release** prerequisites only.

Suggested distribution: Sole iOS engineer — §1 and §3; **operations / finance / IT** — **§2** (Apple, Mac, devices); **backend leadership** — companion backend document; **product and design** — **§1.2–1.5** (architecture, trust boundaries, narrative flow, feature sequences), and **§3** (phased delivery).